

Práctica. CWE-209: Generation of Error Message Containing Sensitive Information.

Instrucciones. Elaborar una aplicación con Node.js que permita ejemplificar la vulnerabilidad CWE-209: Generación de mensajes de error que contienen información confidencial en una aplicación web cliente-servidor.

1. Sigue las instrucciones de la práctica en la siguiente página.
2. Crea un reporte de práctica en un **archivo de Word** con una portada con tu nombre. Guárdalo con el nombre **NombreAlumno_CWE-209.docx**.
3. Coloca **las siguientes capturas de pantalla**, cada una con una **descripción textual** arriba de la captura.
 - a. Coloca la captura de pantalla de tu código fuente en Visual Studio Code del archivo **app.js**.
 - b. Coloca la captura de pantalla de la salida de la página **index.ejs** de autenticación.
 - c. Coloca la captura de pantalla de la salida del error de Node.js donde se vea la api key revelada al atacante.
 - d. Publica tu código fuente en un proyecto público de GitHub. **Coloca la URL** del código fuente publicado en GitHub.
4. Sube tu reporte en un archivo Word a la actividad en Eminus.

Prerrequisitos

1. Esta práctica tiene como prerrequisito tener instalado el software **Visual Studio Code** con la extensión para PHP.
Puede seguir las instrucciones de cómo instalar Visual Studio Code desde la práctica [Instalación de Visual Studio Code](#).
2. Esta práctica tiene como prerrequisito tener instalado la plataforma Node.js en su equipo.
Puede seguir las instrucciones de cómo instalar Node.js desde la práctica [Instalación de Node.js](#).
3. Puede seguir las instrucciones de cómo publicar un proyecto en Visual Studio Code a GitHub desde la práctica [Como publicar un proyecto a GitHub desde Visual Studio Code](#).

Instrucciones

Esta vulnerabilidad consiste en que por medio de una entrada maliciosa, el programa genera un mensaje de error que incluye información confidencial sobre su entorno, usuarios o datos asociados. En este ejemplo, al no realizar un control de excepciones apropiado, permite que un atacante pueda provocar un fallo que revele información valiosa que útil para lanzar otros ataques más graves.

Para este ejemplo, utilizaremos la plataforma Node.js junto con el paquete **Express**, la cual es un web application framework para Node.js mínimo y flexible que proporciona un conjunto sólido de funciones para desarrollar aplicaciones web y móviles. Facilita el rápido desarrollo de aplicaciones web basadas en Nodos. Las siguientes son algunas de las características principales del marco Express:

- Permite configurar middlewares para responder a solicitudes HTTP.
- Define una tabla de enrutamiento que se utiliza para realizar diferentes acciones según el método HTTP y la URL.
- Permite representar dinámicamente páginas HTML basándose en pasar argumentos a plantillas.

En términos del motor de base de datos, usaremos MySQL. Para implementar las vistas se utilizará el motor de plantillas EJS (JavaScript integrado)

A. Crear el programa servidor

1. Asegúrese de contar con una carpeta de trabajo en **C:\codigo**. Si aún no ha creado la carpeta **C:\codigo** hágalo antes de continuar.
2. Seleccione **Archivo > Abrir carpeta** en el menú principal.
3. En el cuadro de diálogo **Abrir carpeta**, cree una carpeta en la ruta **C:\codigo\ps\cwe-209** y selecciónela. Luego haga clic en Seleccionar carpeta.
4. El nombre de la carpeta se convierte en el nombre del proyecto y el nombre del espacio de nombres de forma predeterminada.
5. Abra una Terminal de consola presionando **Ctrl+ñ**.
6. Utilice el comando **npm init** para crear un archivo **package.json** para su aplicación.

```
> npm init
```

7. Este comando le solicita varias cosas, como el nombre y la versión de su aplicación. Por ahora, puedes simplemente presionar **Enter** para aceptar los valores predeterminados para la mayoría de ellos, con la siguiente excepción:

```
entry point: (index.js)
```

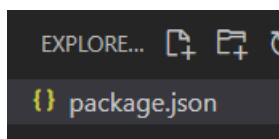
8. Ingrese **app.js**, que será el nombre del archivo principal. En **Node.js** se recomienda que ese sea el nombre del archivo principal.

```
package name: (server)
version: (1.0.0)
description: cwe-476
entry point: (index.js) app.js
test command:
git repository:
keywords:
author:
license: (ISC)
```

```
About to write to C:\codigo\ps\cwe-476\server\package.json:
```

```
{
  "name": "server",
  "version": "1.0.0",
  "description": "cwe-476",
  "main": "app.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "",
  "license": "ISC"
}
Is this OK? (yes)
```

9. Observe que se ha creado un archivo llamado `package.json`. Si quisiera omitir el asistente, solo se agrega la flag `-y` al comando `npm init -y`. Deje abierto este archivo.



10. Ahora instale el paquete **Express** en este directorio y guárdelo en la lista de dependencias del proyecto. Para instalarlo temporalmente y no agregarlo a la lista de dependencias agregue `--no-save`.

```
> npm install express --save
```

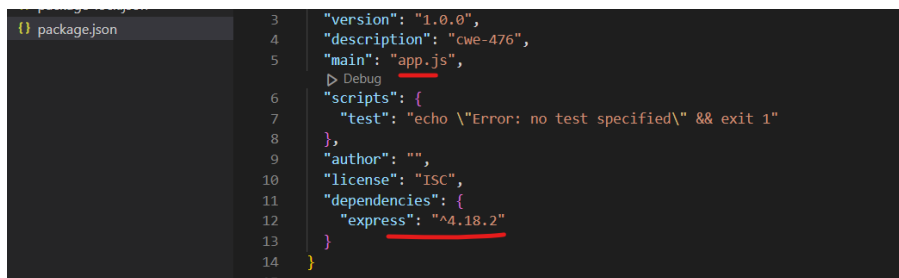
```
PS C:\codigo\ps\cwe-476\server> npm install express

added 62 packages, and audited 63 packages in 3s

11 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\codigo\ps\cwe-476\server> 
```

11. Observe que se ha instalado la versión `4.18.2` de Express.



12. Ahora instale el paquete **ejs** (Embedded Javascript Templating) que es un template engine para Node.js.

```
npm install ej
```

13. También instale el paquete **request** que es un cliente HTTP para Node.js.

```
npm install request
```

14. Por último instale el paquete **express-session** para el manejo de la autenticación de la aplicación.

```
npm install express-session
```

15. Solo como referencia, para desinstalar un paquete, se utiliza **npm uninstall <package_name>**.

16. Cree una nueva carpeta llamada **views**. Ahí es donde colocaremos los templates de HTML que utiliza el *template engine* **ejs**.

17. Dentro de esta carpeta, agregue un archivo llamado **index.ejs** y escriba el siguiente código.

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="icon" href="data:;base64,iVBORw0KGgo=" />
  <title>Programación segura - Universidad Veracruzana</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
</head>

<body class="container mt-4">
  <h2>Programación segura <small class="text-muted fs-5">Universidad Veracruzana</small></h2>

  <form method="post">
    <div class="card mx-auto mt-3">
      <div class="card-header">Inicio de sesión</div>
      <div class="card-body">
        <div class="mb-3">
          <label for="username" class="form-label">Usuario</label>
          <input type="text" class="form-control" name="username" placeholder="Escriba el usuario">
          <%if (locals.username) { %>
            <span class="text-danger">Credenciales incorrectas</span>
          <% } %>
        </div>
        <div class="mb-3">
          <label class="form-label" for="password">Contraseña</label>
          <input class="form-control" type="password" name="password" placeholder="Escriba su contraseña">
        </div>
      </div>
      <div class="card-footer text-center">
        <button id="submit" type="submit" class="btn btn-primary">Iniciar sesión</button>
      </div>
    </div>
  </form>
</body>
</html>
```

18. Observe las etiquetas de **ejs**. Este código será reemplazado en el servidor por código HTML.

```
<%if (locals.username) { %>
  <span class="text-danger">Credenciales incorrectas</span>
<% } %>
```

19. Este archivo es la interfaz principal de nuestro ejemplo. Sería nuestro frontend.

20. Agregue otro archivo llamado **welcome.ejs** que va a ser la página de un usuario autenticado. Coloque el siguiente código.

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="icon" href="data:;base64,iVBORw0KGgo=" />
```

```
<title>Programación segura - Universidad Veracruzana</title>
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
</head>

<body class="container mt-4">
  <h2>Programación segura <small class="text-muted fs-5">Universidad Veracruzana</small></h2>

  <div class="h-100 mt-5 p-5 bg-body-tertiary border rounded-3">
    <h2>Sitio protegido</h2>
    <p>Bienvenido <strong><%= locals.username %></strong> a este sitio protegido por correo electrónico y un código de
    acceso.</p>
    <a href="/" class="btn btn-outline-secondary" type="button">Cerrar sesión</a>
  </div>
</body>

</html>
```

21. Ahora vamos a crear el backend en Node.js. Cree un nuevo archivo llamado **app.js** y agregue el siguiente código.

```
const express = require('express')
const request = require('request');
const session = require('express-session')

const app = express()
app.use(session({ secret: 'miappsegura', resave: false, saveUninitialized: true }))
const port = 3000

//app.use(express.static("public"))
app.use(express.urlencoded({ extended: true }))
app.set('view engine', 'ejs')

let api = {
  // Esta key nos da acceso a toda la API. No compartas esta llave con nadie!
  key: "88665751-288d-4175-852f-6519d79fdf1f"
}

app.get('/', (req, res) => {
  req.session.destroy()
  res.render('index')
})

app.get('/welcome', (req, res) => {
  if (req.session.username) {
    res.render('welcome', { username: req.session.username })
  } else {
    res.send('No esta autorizado para ver este contenido.')
  }
})

app.post('/', (req, res) => {
  // Obtenemos el dominio en el que el usuario nos visita
  // pues tenemos varios en el cluster
  let host = req.get('host')
  let username = req.body.username
  let password = req.body.password

  // Verificamos que este operacional
  if (request(`http://${host}/codigos?api_key=${api.key}`)) {
    console.log('Funcionando')
  } else {
    console.log('No esta funcionando la API')
    return res.render('index', { username: 'username' })
  }

  // Contactamos a nuestra API para obtener los usuarios
  request(`http://${host}/codigos?api_key=${api.key}`, function (error, response, body) {
    let users = JSON.parse(body)
    var user = users.find(u => u.name === username && u.code === password); // encuentra el usuario.
    if (user) {
      req.session.username = username
      res.redirect('/welcome')
    } else {
      res.render('index', { username: 'username' })
    }
  })
})
```

```
})  
  
// Cuidado con esta, solo se puede ver con la api-key  
app.get('/codigos', (req, res) => {  
  // Validamos que tenga permiso de leer esto  
  if (req.query.api_key == api.key)  
    res.json(codigos)  
  else  
    res.status(401).send('Sin autorización')  
})  
  
app.listen(port, () => {  
  console.log(`Aplicación de ejemplo escuchando en el puerto ${port}`)  
})  
  
var codigos = [  
  { name: 'admin', code: "patito" },  
  { name: 'user', code: "34fiufdeQ@5" },  
  { name: 'guest', code: "uyy8787##$JK" }  
];
```

22. Esta aplicación inicia un servidor web y escucha las conexiones en el puerto **3000**.

23. Su proyecto debería tener los siguientes archivos.

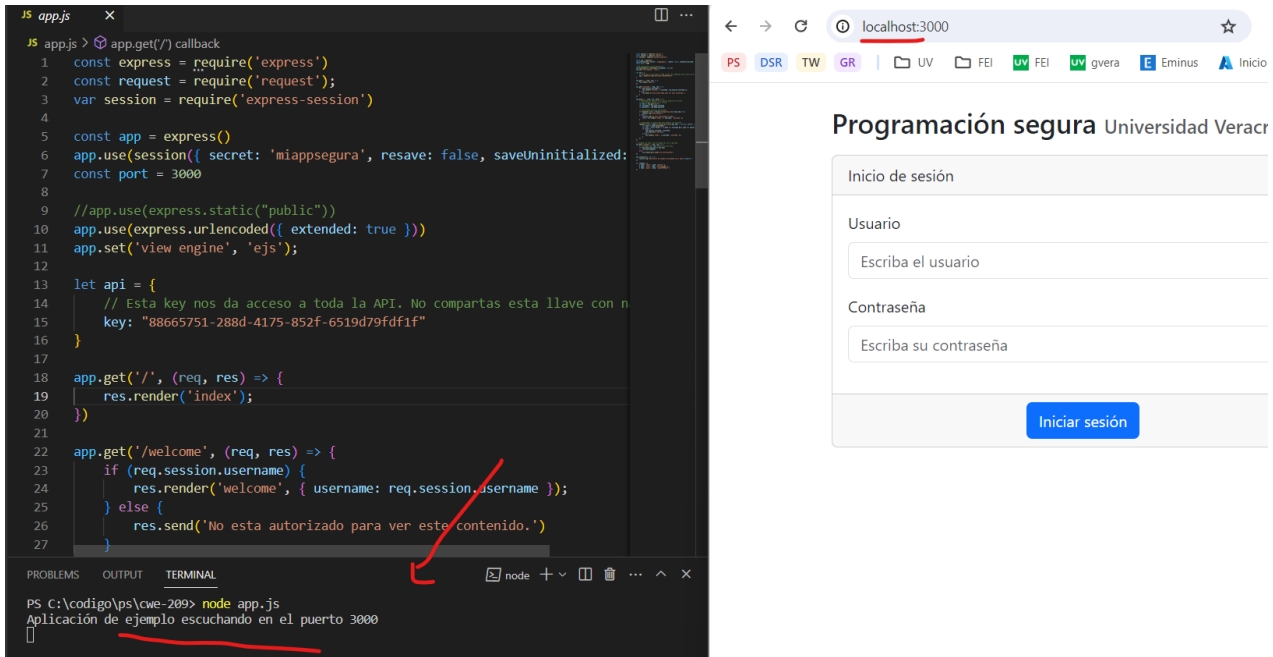
```
> node_modules  
▼ views  
  <> index.ejs  
  <> welcome.ejs  
JS app.js  
{ package-lock.json  
{ package.json
```

24. Ejecute la aplicación con el siguiente comando desde la terminal de VS Code.

```
> node app.js
```

```
PS C:\codigo\ps\cwe-476> node app.js  
Aplicación de ejemplo escuchando en el puerto 3000
```

25. Abra un navegador web y abra la ruta <http://localhost:3000/> para ver el resultado.



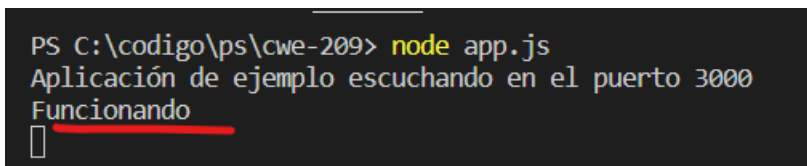
26. Escriba algún nombre de usuario y contraseña incorrecta y presione **Iniciar sesión**.

Usuario

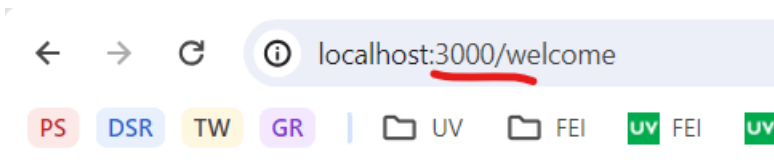
Escriba el usuario

Credenciales incorrectas

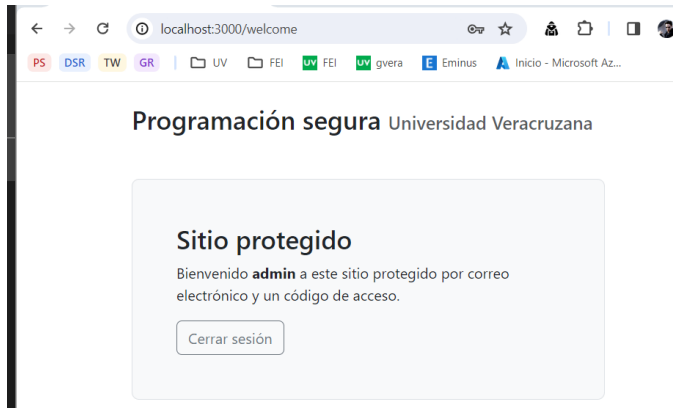
27. Observe como la aplicación le valida que el usuario sea correcto. También vea en la consola de VS Code, que la aplicación válida el funcionamiento de la API.



28. Intente entrar a la ruta <http://localhost:3000/welcome>.



29. Muy bien. Regrese a la página de logueo e intente entrar con una cuenta correcta como usuario **admin** y contraseña **patito**.



30. Si inspecciona el sitio con las herramientas del navegador, verá que no estamos enviando nada revelador por ningún método http. La aplicación se mira segura aparentemente.
31. Enhorabuena. Ha creado su aplicación de manera correcta.

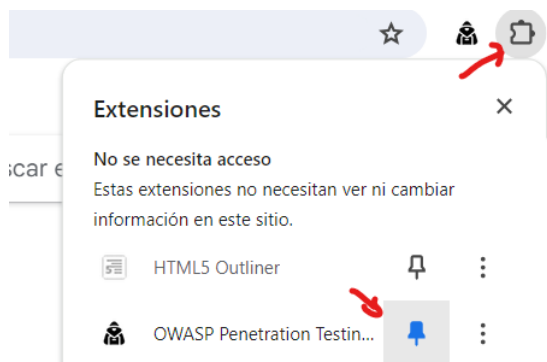
B. Explotar la vulnerabilidad

En este ejemplo vamos a encontrar el punto de entrada de la vulnerabilidad en el manejo de los errores.

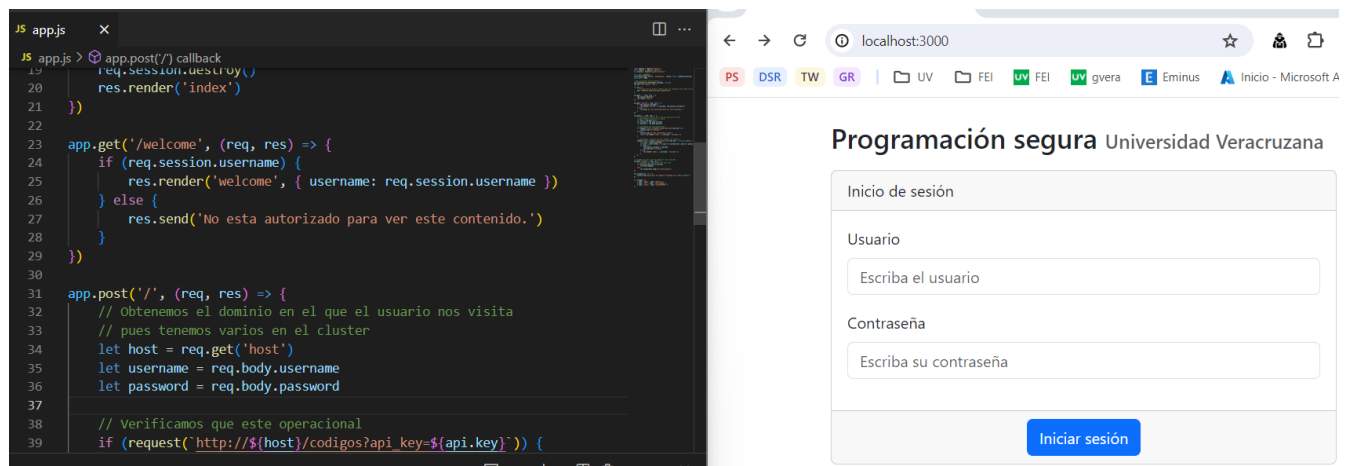
1. Abra su navegador **Chrome** e instale la extensión desde <https://chromewebstore.google.com/detail/ojkchikaholjmcnefhjlbohackpeeknd>.



2. Asegúrese de tenerlo activado haciendo clic en el pin de la barra de Chrome.



3. Abra VS Code ejecutando su servidor de Node.js del lado izquierdo de su pantalla. Abra del lado derecho su aplicación web corriendo en <http://localhost:3000/>.

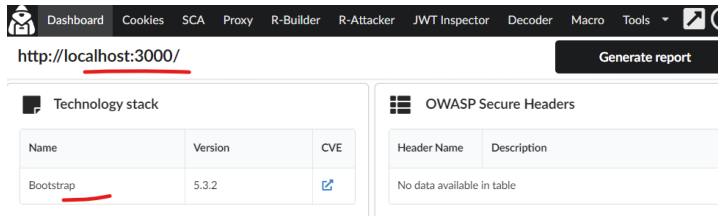


4. Ingrese un usuario no válido como usuario: **nosoyusuario** y contraseña: **novalida**.
5. Observe que no puede entrar al apartado.

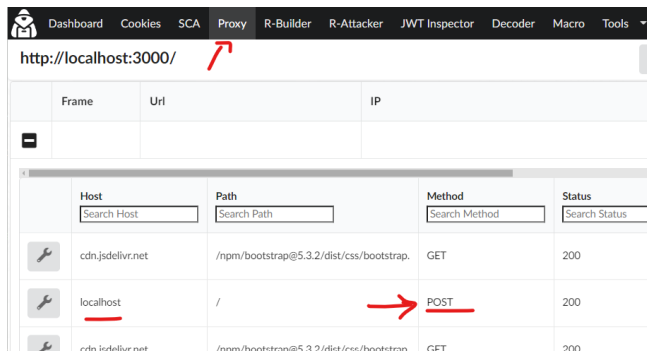
Escriba el usuario

Credenciales incorrectas

6. Haga clic en la extensión **OWASP** y verifique que está recopilando datos. Si es necesario recargue la página para que se intercepten los datos.



7. Vaya a la pestaña **Proxy**. Revise las peticiones hasta que vea una con método POST. Si no la hubiera, vuelva a intentar logearse.



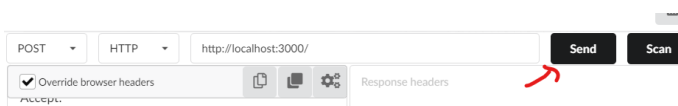
8. Haga clic en el botón de la llave de herramienta del lado izquierdo. En esta ventana se capturó una solicitud y nosotros podemos enviarla nuevamente, pero alterando los datos.
9. Del lado izquierdo vaya hasta la parte de abajo. Observe como aparecen los datos que acabamos de ingresar. Es el cuerpo de la petición enviado hacia el servidor.

```
41.0.0.0;  
_ga_N5QSTR1E5G=GS1.1.1706253027.3.1.170625344  
4.0.0.0; connect.sid=s%3AOeQUSMZe-  
kvGCUgUSeJcs1XRwdFtwDM.9ysRjBKfupIX4kbHHoY  
UwZRMrkJhplTkUzcBv8gGVRs  
Cache-Control: no-cache, no-store  
Host: localhost:3000  
Content-Length: 39  
  
password=novalida&username=nosoyusuario
```

10. No conocemos la contraseña correcta, por lo que de nada nos sirve alterar esos datos.
11. Una práctica muy común entre los atacantes, es alterar los encabezados de las peticiones y observar cómo responden los servidores. Así que vamos a alterar el valor de **Host: localhost:3000**. Borre el valor de este encabezado de la solicitud. Déjelo como **Host:**.

```
UwZRMrkJhplTkUzcBv8gGVRs  
Cache-Control: no-cache, no-store  
Host:  
Content-Length: 39  
  
password=novalida&username=nosoyusuario
```

12. Ahora presione el botón negro que dice **Send**.



13. Algo sucedió! Vea la consola de VS Code.

```
Funcionando
Funcionando
Error: Invalid URI "http://codigos?api_key=88665751-288d-4175-852f-6519d79fdf1f"
  at Request.init (C:\codigo\ps\cwe-209\node_modules\request\request.js:273:31)
  at new Request (C:\codigo\ps\cwe-209\node_modules\request\request.js:127:8)
  at request (C:\codigo\ps\cwe-209\node_modules\request\index.js:53:10)
  at C:\codigo\ps\cwe-209\app.js:39:9
  at Layer.handle [as handle_request] (C:\codigo\ps\cwe-209\node_modules\express\lib\router\layer.js:95:5)
  at next (C:\codigo\ps\cwe-209\node_modules\express\lib\router\route.js:144:13)
  at Route.dispatch (C:\codigo\ps\cwe-209\node_modules\express\lib\router\route.js:114:3)
  at Layer.handle [as handle_request] (C:\codigo\ps\cwe-209\node_modules\express\lib\router\layer.js:95:5)
  at C:\codigo\ps\cwe-209\node_modules\express\lib\router\index.js:284:15
  at Function.process_params (C:\codigo\ps\cwe-209\node_modules\express\lib\router\index.js:346:12)
```

14. Ocurrió un error en el servidor y nos está enviando detalles del error. ¿Lo habrá enviado también al cliente?

```
settings-time-1=10Y4/34/44;
_ga_6N3GNEW55S=GS1.1.1700786554.7.1.17007867
41.0.0.0;
_ga_N5QSTR1E5G=GS1.1.1706253027.3.1.170625344
4.0.0.0; connect.sid=s%3AOeQUSMZ-
kvGCUgUSeJcs1XRewdFtwDM.9ysRjBKfuplX4kbHHoY
UwZRMrkJhplTkUzcBv8gGVRs
Cache-Control: no-cache, no-store
Host:
Content-Length: 39

password=novalida&username=nosoyusuario

<meta charset="utf-8">
<title>Error</title>
</head>
<body>
<pre>Error: Invalid URI &quot;http://codigos?
api_key=88665751-288d-4175-852f-6519d79fdf1f&quot;<br>
&nbsp; &nbsp; &nbsp;at Request.init (C:\codigo\ps\cwe-
209\node_modules\request\request.js:273:31)<br> &nbsp; &nbsp;
&nbsp; &nbsp;at new Request (C:\codigo\ps\cwe-
209\node_modules\request\request.js:127:8)<br> &nbsp; &nbsp;
&nbsp; &nbsp;at request (C:\codigo\ps\cwe-
209\node_modules\request\index.js:53:10)<br> &nbsp; &nbsp; &nbsp; &nbsp;at
C:\codigo\ps\cwe-209\app.js:39:9<br> &nbsp; &nbsp; &nbsp; &nbsp;at
Layer.handle [as handle_request] (C:\codigo\ps\cwe-
```

15. Así es. Ahora el cliente puede ver una clave de api y una ruta revelada en el error. Copie y pegue esa ruta en otra pestaña de Chrome. Agregue el nombre de host que eliminamos de la petición.

http://localhost:3000/codigos/?api_key=88665751-288d-4175-852f-6519d79fdf1f

16. Aparecen todos los nombres y contraseñas de acceso de los usuarios del sistema.

```
localhost:3000/codigos/?api_key=88665751-288d-4175-852f-6519d79fdf1f
PS DSR TW GR | UV FEI UV FEI UV gvera E Eminus Inicio - Microsoft Az... GitHub
[{"name":"admin","code":"patito"}, {"name":"user","code":"34fiufdeQ@5"}, {"name":"guest","code":"uyy8787###$JK"}]
```

17. Hay un grave fallo en la aplicación debido a un incorrecto manejo de los errores.

18. Aunque esto es una demostración, este tipo de fallos son muy comunes en la programación de sistemas y los atacantes lo saben. Lo que se realiza es utilizar **botnets** para estar enviando solicitudes incompletas a sitios al azar en la red Y cuando detectan que algún sitio responde de manera distinta entre una solicitud completa y otra que no, es candidato a un ataque más riguroso.

19. Felicidades, ha creado su aplicación de manera correcta.